React.js

Complete Study Notes
with Spring Boot + Axios Integration

- ✥ Components & Templates
- ✥ useState Hook
- ✥ Props & Reusability
- ✥ useEffect Hook
- ✥ Fetching Data with Axios
- ✥ React Router
- ✥ Route Parameters
- ✥ useEffect Cleanup
- ✥ Error Handling

Spring Boot + Axios backend integration

01 Components & Templates

In React, a **Component** is an independent, reusable piece of UI. Every React app is a tree of components. Each component is essentially a JavaScript function that returns **JSX** — a syntax that looks like HTML but is actually JavaScript under the hood.

- Components must start with a **Capital Letter** (e.g., App, Navbar).
- Each component returns a single root JSX element.
- **JSX** lets you write UI structure directly in JavaScript.
- Components are exported so other files can import and reuse them.

JSX

```
// App.js — the root component
function App() {
  return (
    <div className='App'>
      <div className='content'>
        <h1>App Component</h1>
      </div>
    </div>
  );
}

export default App;
```

Why className instead of class?

JSX compiles to JavaScript. Since **class** is a reserved keyword in JS, React uses **className** instead. Under the hood, React converts it to the correct HTML **class** attribute.

02 Dynamic Values in Templates

JSX allows you to embed **any JavaScript expression** using curly braces **{ }**. This is how you display dynamic data — variables, calculations, strings, arrays — inside your UI.

- Strings, numbers, arrays — all render as text.
- JavaScript expressions like Math.random() evaluate live.
- Objects cannot be rendered directly — React will throw an error.
- Use curly braces for dynamic attribute values too (e.g., href={'{link}'}).
- Comments in JSX: {/* comment here */}

JSX

```
function App() {
  const title = 'Welcome to the new blog';
  const likes = 50;
  const link = 'http://www.google.com';

  return (
    <div className='App'>
      <div className='content'>
        <h1>{ title }</h1>
        <p>Liked { likes } times</p>
        <p>{ 10 }</p>
        <p>{ 'hello, ninjas' }</p>
        <p>{ [1, 2, 3, 4, 5] }</p>
        <p>{ Math.random() * 10 }</p>
        <a href={link}>Google Site</a>
      </div>
    </div>
  );
}
```

What can go inside { }?

■ Strings, numbers, arrays, expressions, function calls, ternary operators

■ Objects like { name: 'yoshi' } — React cannot render plain objects. Render a specific property instead: { person.name }

03 Multiple Components

Real apps are built from many components working together. You define each component as its own function (or in its own file), then compose them inside a parent component using JSX tags.

- Components can be in the same file or separate files (recommended).
- Use **import/export** to share components across files.
- Nest components using self-closing or paired JSX tags.

JSX

```
// App.js
function App() {
  return (
    <div className='App'>
      <Navbar />
      <div className='content'>
        <Home />
      </div>
    </div>
  );
}

// Navbar component
const Navbar = () => {
  return (
    <nav className='navbar'>
      <h1>The Dojo Blog</h1>
      <div className='links'>
        <a href='/'>Home</a>
        <a href='/create'>New Blog</a>
      </div>
    </nav>
  );
};

// Home component
const Home = () => {
  return (
    <div className='home'>
      <h2>Homepage</h2>
    </div>
```

```
    );
  };
```

04 Adding Styles

React supports two main ways to style components: **Global CSS via import** and **Inline styles**. For most projects you start with a global stylesheet, then use inline styles for dynamic or one-off overrides.

Global CSS Import

JSX

```
// At the top of App.js or index.js
import './index.css';
```

This applies the CSS globally to your entire app, just like linking a stylesheet in HTML. Class names are applied via **className**.

Inline Styles

Inline styles in JSX are written as a **JavaScript object** (not a plain string). Property names are camelCased.

JSX

```
const Navbar = () => {
  return (
    <nav className='navbar'>
      <h1>The Dojo Blog</h1>
      <div className='links'>
        <a href='/'>Home</a>
        <a
          href='/create'
          style={{
            color: 'white',
            backgroundColor: '#f1356d',
            borderRadius: '8px'
          }}
        >
          New Blog
        </a>
      </div>
    </nav>
  );
};
```

Double curly braces {{ }}?

The outer **{ }** are JSX expression delimiters. The inner **{ }** are a JavaScript object literal. So `style={{ color: 'red' }}` means "pass this JS object as the style prop".

05 Click Events

React handles events using **camelCase** event names as JSX props (e.g., **onClick**, **onChange**). You pass a function reference — not a function call — as the value.

- **onClick={handleClick}** — passes the function itself (correct ■).
- **onClick={handleClick()}** — calls it immediately on render (wrong ■).
- To pass arguments, wrap in an arrow function: onClick={() => handleFn(arg)}.
- The event object **e** is available as a parameter and gives access to **e.target**, etc.

JSX

```jsx
const Home = () => {

  const handleClick = (e) => {
    console.log('hello ninjas', e);
  };

  const handleClickAgain = (name, e) => {
    console.log('hello ' + name, e.target);
  };

  return (
    <div className='home'>
      <h2>Homepage</h2>
      <button onClick={handleClick}>
        Click me
      </button>
      <button onClick={(e) => handleClickAgain('mario', e)}>
        Click me again
      </button>
    </div>
  );
};
```

06 Using State (useState Hook)

React's **useState** hook lets a component remember values between renders. When state changes, React automatically **re-renders** the component with the new value. Directly mutating a variable does NOT trigger a re-render — you must use the setter.

SYNTAX

```
const [value, setValue] = useState(initialValue);
// ^ ^ ^
// current setter fn initial value
```

JSX

```
import { useState } from 'react';

const Home = () => {
  const [name, setName] = useState('mario');
  const [age, setAge] = useState(25);

  const handleClick = () => {
    setName('luigi'); // triggers re-render
    setAge(30);
  };

  return (
    <div className='home'>
      <h2>Homepage</h2>
      <p>{ name } is { age } years old</p>
      <button onClick={handleClick}>Click me</button>
    </div>
  );
};
```

Why not just use let?

Changing a **let** variable doesn't tell React anything changed, so the UI won't update. **useState** wires the value into React's rendering system — calling the setter schedules a re-render with the new value.

07 Outputting Lists

To render a list of items in React, use the JavaScript **.map()** method. Each element returned by map becomes a JSX node. React requires a unique **key** prop on each list item so it can efficiently track and update individual elements.

- Use **.map()** to transform an array into JSX elements.
- Always add a **key** prop — use a unique id, not the array index if avoidable.
- The key must be unique *among siblings*, not globally.

JSX

```jsx
import { useState } from 'react';

const Home = () => {
  const [blogs, setBlogs] = useState([
    { title: 'My new website', author: 'mario', id: 1 },
    { title: 'Welcome party!', author: 'yoshi', id: 2 },
    { title: 'Web dev top tips', author: 'mario', id: 3 },
  ]);

  return (
    <div className='home'>
      {blogs.map(blog => (
        <div className='blog-preview' key={blog.id}>
          <h2>{ blog.title }</h2>
          <p>Written by { blog.author }</p>
        </div>
      ))}
    </div>
  );
};

export default Home;
```

08 Props

Props (short for properties) are the mechanism for passing data from a **parent component** to a **child component**. Think of them as function arguments — you pass values in, the child reads them.

Why are props useful? Without props, every component would need to hardcode its own data. Props make components reusable and flexible — the same component can display different content depending on what the parent passes in.

- Props are passed as JSX attributes: blogs={'{blogs}'}.
- The child receives all props as a single object, typically destructured: ({ blogs, title }).
- Props are **read-only** — a child must never modify its props.
- **Functions can be passed as props too** (covered next section).

JSX

```
// Home.js – parent passes data down
import BlogList from './BlogList';

const Home = () => {
  const [blogs, setBlogs] = useState([
    { title: 'My new website', author: 'mario', id: 1 },
    { title: 'Welcome party!', author: 'yoshi', id: 2 },
    { title: 'Web dev top tips', author: 'mario', id: 3 },
  ]);

  return (
    <div className='home'>
      <BlogList blogs={blogs} title='All Blogs' />
    </div>
  );
};
```

JSX

```
// BlogList.js – child receives props
const BlogList = ({ blogs, title }) => {
  return (
    <div className='blog-list'>
      <h2>{ title }</h2>
      {blogs.map(blog => (
        <div className='blog-preview' key={blog.id}>
          <h2>{ blog.title }</h2>
```

```
            <p>Written by { blog.author }</p>
          </div>
        ))}
      </div>
    );
  };
```

09 Reusing Components & Functions as Props

One of React's biggest strengths is **component reusability**. The same component can be rendered multiple times with different props. You can also pass **functions as props**, enabling child components to communicate back to their parent — for example, telling the parent to delete an item.

- Render the same component with different data by changing the props passed.
- Pass a handler function (defined in the parent) as a prop.
- The child calls it with the relevant data (e.g., an id to delete).
- This pattern is called **lifting state up** — the parent owns the state, child triggers changes.

JSX

```
// Home.js — owns state, passes handler down
const Home = () => {
  const [blogs, setBlogs] = useState([...]);

  const handleDelete = (id) => {
    const newBlogs = blogs.filter(blog => blog.id !== id);
    setBlogs(newBlogs);
  };

  return (
    <div className='home'>
      {/* Reuse BlogList with different filters */}
      <BlogList blogs={blogs} title='All Blogs'
                handleDelete={handleDelete} />
      <BlogList blogs={blogs.filter(b => b.author === 'mario')}
                title='Mario's Blogs'
                handleDelete={handleDelete} />
    </div>
  );
};
```

JSX

```
// BlogList.js — calls the handler passed from parent
const BlogList = ({ blogs, title, handleDelete }) => {
  return (
    <div className='blog-list'>
      <h2>{ title }</h2>
      {blogs.map(blog => (
```

```
          <div className='blog-preview' key={blog.id}>
            <h2>{ blog.title }</h2>
            <p>Written by { blog.author }</p>
            <button onClick={() => handleDelete(blog.id)}>
              Delete
            </button>
          </div>
        ))}
      </div>
    );
  };
```

10 useEffect Hook — The Basics

useEffect lets you run **side effects** in function components — things that happen outside the normal render cycle, such as fetching data, setting up subscriptions, or manually changing the DOM.

It takes two arguments: a **callback function** (the effect), and an optional **dependency array** that controls when the effect re-runs.

Syntax	When it runs
`useEffect(() => { ... })`	After EVERY render (any state change)
`useEffect(() => { ... }, [])`	Only ONCE — on initial mount
`useEffect(() => { ... }, [name])`	On mount AND whenever 'name' changes

1. No dependency array — runs after every render

JSX

```
useEffect(() => {
  console.log('runs after every render');
  console.log(blogs);
});
// Fires after initial render AND after every state update
```

2. Empty array [ ] — runs only once

JSX

```
useEffect(() => {
  console.log('runs only on mount');
}, []);
// Perfect for initial data fetching
```

3. Dependency array [name] — runs when variable changes

JSX

```
useEffect(() => {
  console.log('runs when name changes');
  console.log(blogs);
}, [name]);
// Also runs once on initial mount

// Changing name triggers the effect:
```

```
<button onClick={() => setName('luigi')}>change name</button>
```

11 Fetching Data with useEffect + Axios

Instead of the browser's **fetch** API, we'll use **Axios** — a popular HTTP client that works great with Spring Boot backends. Axios automatically parses JSON responses and provides cleaner error handling.

Install Axios

TERMINAL

```
npm install axios
```

Fetching blogs from Spring Boot backend

JSX

```
import { useEffect, useState } from 'react';
import axios from 'axios';
import BlogList from './BlogList';

const Home = () => {
  const [blogs, setBlogs] = useState(null);

  useEffect(() => {
    // GET http://localhost:8080/blogs
    axios.get('http://localhost:8080/blogs')
      .then(response => {
        // axios wraps data in response.data
        setBlogs(response.data);
      });
  }, []); // [] = only fetch once on mount

  return (
    <div className='home'>
      {blogs && <BlogList blogs={blogs} />}
    </div>
  );
};
```

fetch vs axios

fetch: response.json() needed to parse, no auto error on 4xx/5xx.

axios: response.data is already parsed JSON, throws errors on non-2xx status automatically. Much cleaner with Spring Boot REST APIs.

12 Conditional Loading Message

Network requests take time. It's good UX to show a loading indicator while waiting for the data. Use a **isPending** state variable — set it to **true** before the request and **false** once data arrives.

- Initialize **isPending** as **true** (we start in a loading state).
- After data loads, set **isPending** to **false**.
- Render the loading message conditionally using **&&**.

JSX

```
import { useEffect, useState } from 'react';
import axios from 'axios';
import BlogList from './BlogList';

const Home = () => {
  const [blogs, setBlogs] = useState(null);
  const [isPending, setIsPending] = useState(true);

  useEffect(() => {
    axios.get('http://localhost:8080/blogs')
      .then(response => {
        setBlogs(response.data);
        setIsPending(false); // hide loading spinner
      });
  }, []);

  return (
    <div className='home'>
      { isPending && <div>Loading...</div> }
      { blogs && <BlogList blogs={blogs} /> }
    </div>
  );
};
```

The && shortcut

In JSX, **{condition && <Component />}** renders the component only when the condition is truthy. It's equivalent to: if (condition) { return <Component /> }

13 Handling Fetch Errors

Errors can happen for two reasons: the server responds with an error status (4xx/5xx), or the network fails entirely. With **Axios + Spring Boot**, both cases are caught cleanly in a **.catch()** block — because Axios automatically throws on non-2xx HTTP status codes.

- **axios.get(...).catch(err)** catches both network errors and HTTP errors.
- **err.response** contains the server's response (status, data) for HTTP errors.
- **err.message** gives a human-readable error string.
- Store the error in state and render it conditionally.

JSX

```
import { useEffect, useState } from 'react';
import axios from 'axios';

const Home = () => {
  const [blogs, setBlogs] = useState(null);
  const [isPending, setIsPending] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    axios.get('http://localhost:8080/blogs')
      .then(response => {
        setBlogs(response.data);
        setIsPending(false);
        setError(null);
      })
      .catch(err => {
        // Axios throws on 4xx/5xx AND network failures
        setIsPending(false);
        setError(err.message);
      });
  }, []);

  return (
    <div className='home'>
      { error && <div className='error'>{ error }</div> }
      { isPending && <div>Loading...</div> }
      { blogs && <BlogList blogs={blogs} /> }
    </div>
  );
```

```
};
```

Custom useFetch Hook with Axios

To avoid repeating fetch logic in every component, extract it into a **custom hook**. By convention, custom hooks start with **use**.

JSX

```
// hooks/useFetch.js
import { useState, useEffect } from 'react';
import axios from 'axios';

const useFetch = (url) => {
  const [data, setData] = useState(null);
  const [isPending, setIsPending] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    axios.get(url)
      .then(response => {
        setData(response.data);
        setIsPending(false);
        setError(null);
      })
      .catch(err => {
        setIsPending(false);
        setError(err.message);
      });
  }, [url]); // re-fetch if url changes

  return { data, isPending, error };
};

export default useFetch;
```

Usage in any component:

JSX

```
import useFetch from './hooks/useFetch';

const Home = () => {
  const { data: blogs, isPending, error } =
```

```
    useFetch('http://localhost:8080/blogs');

  return (
    <div className='home'>
      { error && <div>{ error }</div> }
      { isPending && <div>Loading...</div> }
      { blogs && <BlogList blogs={blogs} /> }
    </div>
  );
};
```

14 The React Router

React is a **Single Page Application (SPA)** framework — by default there's only one HTML page. **React Router** intercepts URL changes and renders the correct component without a full page reload, giving users the feel of traditional multi-page navigation.

Install React Router

TERMINAL

```
npm install react-router-dom
```

Core concepts

- **BrowserRouter** (aliased as Router) — wraps the entire app, enables routing.
- **Switch** — renders only the first matching route.
- **Route** — maps a URL path to a component.
- **Link** — replaces <a href> for in-app navigation (no page reload).

JSX

```
// App.js
import { BrowserRouter as Router, Route, Switch } from 'react-router-dom';
import Navbar from './Navbar';
import Home from './Home';
import Create from './Create';

function App() {
  return (
    <Router>
      <div className='App'>
        <Navbar />
        <div className='content'>
          <Switch>
            <Route exact path='/'>
              <Home />
            </Route>
            <Route path='/create'>
              <Create />
            </Route>
          </Switch>
        </div>
      </div>
```

```
      </Router>
    );
  }
```

15 Exact Match Routes

By default, React Router matches routes using **prefix matching**. The path **/** matches *every* URL because every URL starts with **/**. Without **exact**, both **/** and **/create** would render the Home component.

- Without **exact**: **/create** matches both **/** AND **/create**.
- With **exact**: **/** only matches when the path is *exactly* **/**.
- **Switch** stops at the first match — order matters!

JSX

```
// Without exact — BUG: Home renders for all routes
<Switch>
  <Route path='/'> {/* matches /create too! */}
    <Home />
  </Route>
  <Route path='/create'>
    <Create />
  </Route>
</Switch>

// With exact — CORRECT
<Switch>
  <Route exact path='/'> {/* only matches exactly '/' */}
    <Home />
  </Route>
  <Route path='/create'>
    <Create />
  </Route>
</Switch>
```

16 Router Links

Regular **<a href>** tags cause a full page reload — the browser fetches the HTML from scratch, losing all React state. React Router's **<Link>** component intercepts the click and updates the URL using the History API, triggering React Router to swap components *without a reload*.

- Import **Link** from **react-router-dom**.
- Use **to='...'** instead of **href='...'**.
- Link accepts the same children and style props as a normal anchor.
- The page never reloads — React state is preserved.

JSX

```jsx
import { Link } from 'react-router-dom';

const Navbar = () => {
  return (
    <nav className='navbar'>
      <h1>The Dojo Blog</h1>
      <div className='links'>
        {/* Link instead of <a> — no page reload */}
        <Link to='/'>Home</Link>
        <Link
          to='/create'
          style={{
            color: 'white',
            backgroundColor: '#f1356d',
            borderRadius: '8px'
          }}
        >
          New Blog
        </Link>
      </div>
    </nav>
  );
};
```

Link vs NavLink

Link — basic navigation, no active-state styling.

NavLink — same as Link but automatically adds an **active** CSS class when the current URL matches the **to** path. Great for highlighting the active menu item in a navbar.

17 useEffect Cleanup

When a component **unmounts** (is removed from the DOM) while an async operation is still running, React will warn about setting state on an unmounted component. The solution is to **cancel** the request in the useEffect **cleanup function**.

With Axios, the standard approach is to use an **AbortController** (or an Axios CancelToken). When the component unmounts, the cleanup function fires and aborts the in-flight request.

- The cleanup function is what you **return** from useEffect.
- It runs when the component unmounts, or before the effect re-runs.
- With Axios: pass **signal: controller.signal** to the request config.
- In cleanup: call **controller.abort()**.
- Check for abort in catch: axios.isCancel(err) returns true.

JSX

```
// hooks/useFetch.js — with cleanup
import { useState, useEffect } from 'react';
import axios from 'axios';

const useFetch = (url) => {
  const [data, setData] = useState(null);
  const [isPending, setIsPending] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const controller = new AbortController();

    axios.get(url, { signal: controller.signal })
      .then(response => {
        setData(response.data);
        setIsPending(false);
        setError(null);
      })
      .catch(err => {
        if (axios.isCancel(err)) {
          console.log('Request cancelled:', err.message);
        } else {
          setIsPending(false);
          setError(err.message);
        }
```

```
      });

    // Cleanup: runs on unmount or before next effect
    return () => controller.abort();

  }, [url]);

  return { data, isPending, error };
};

export default useFetch;
```

When does cleanup run?

1. When the component is **removed from the DOM** (user navigates away).
2. Before the effect re-runs (e.g., the **url** dependency changed).

This prevents memory leaks and 'Can't perform a React state update on an unmounted component' warnings.

18 Route Parameters

Route parameters let you embed **dynamic values** in the URL path. For example, **/blogs/3** could show the blog with ID 3. You define a parameter with a colon prefix in the route path, then read it in the component using the **useParams** hook.

Step 1 — Define the route with a parameter

JSX

```
// App.js
<Route path='/blogs/:id'>
  <BlogDetails />
</Route>
// :id is a URL parameter — it can be any value
```

Step 2 — Link to parameterized URLs

JSX

```
import { Link } from 'react-router-dom';

const BlogList = ({ blogs }) => {
  return (
    <div className='blog-list'>
      {blogs.map(blog => (
        <div key={blog.id}>
          {/* Template literal builds the URL */}
          <Link to={`/blogs/${blog.id}`}>
            <h2>{ blog.title }</h2>
            <p>Written by { blog.author }</p>
          </Link>
        </div>
      ))}
    </div>
  );
};
```

Step 3 — Read the parameter with useParams

JSX

```
import { useParams } from 'react-router-dom';
import useFetch from './hooks/useFetch';
```

```
const BlogDetails = () => {
  const { id } = useParams();
  // id = '3' when URL is /blogs/3

  const { data: blog, isPending, error } =
    useFetch(`http://localhost:8080/blogs/${id}`);

  return (
    <div className='blog-details'>
      { isPending && <div>Loading...</div> }
      { error && <div>{ error }</div> }
      { blog && (
        <article>
          <h2>{ blog.title }</h2>
          <p>Written by { blog.author }</p>
          <div>{ blog.body }</div>
        </article>
      )}
    </div>
  );
};
```

Query Parameters (optional key-value pairs)

Beyond path parameters, URLs support **query strings** — optional key-value pairs after a **?**. These are not defined in the route path — they're free-form. React Router provides **useLocation** and **URLSearchParams** to read them.

JSX

```
// URL examples:
// /blogs?author=mario
// /blogs?author=mario&sort=date
// /blogs?page=2&limit=10

import { useLocation } from 'react-router-dom';

const Home = () => {
  const location = useLocation();
  // location.search = '?author=mario&sort=date'

  const params = new URLSearchParams(location.search);
```

```
  const author = params.get('author'); // 'mario'
  const sort = params.get('sort'); // 'date'
  const page = params.get('page'); // null if not present

  return (
    <div>
      <p>Filtering by author: { author }</p>
    </div>
  );
};
```

With Spring Boot, query params are read via **@RequestParam** on the backend and sent through Axios like this:

JSX

```
// Axios with query params — Spring Boot backend
axios.get('http://localhost:8080/blogs', {
  params: {
    author: 'mario',
    sort: 'date',
    page: 2,
  }
})
// Axios serializes this to: /blogs?author=mario&sort=date&page=2
```

URL Type	Example	How to read
Path param	`/blogs/:id → /blogs/3`	`useParams()`
Query param	`/blogs?author=mario`	`useLocation() + URLSearchParams`
Both combined	`/blogs/3?highlight=true`	`useParams() + useLocation()`